

## Assumptions per Entity

### *dim\_date*

- Represents a calendar date that is derived from FL\_DATE
- date\_id is a key with the format YYYYMMDD
  - Will aid in joins and grouping
- One date can be referenced by many flights

### *dim\_airport*

- Represents an airport from T\_MASTER\_CORD and is identified by airport\_id
- Airport location (latitude and longitude) and other geographic info will be here
- airport\_wac will map the airport to a WAC region
  - For future reference WAC = World Area Code according to the Bureau of Transportation

### *dim\_carrier*

- Represents an airline “carrier” from T\_CARRIER\_DECODE and is identified by airline\_id
- Carrier names, region, and group will be the attributes that describe each airline
  - Shouldn’t repeat per flight

### *dim\_wac*

- Represents the WAC geography codes from T\_WAC\_COUNTRY\_STATE
- Used by airports and airline carriers and stored for flights to quickly distinguish flights by region

### *dim\_delay\_type*

- Define delay categories
  - Carrier, Weather, NAS, Security, Late Aircraft

### *fact\_flight*

- Grain: one row per flight instance from T\_ONTIME\_MARKETING
  - 600,000 entires
- Should contain only foreign keys, measures, and flags

### *fact\_flight\_delay*

- This acts as a bridge
  - Each row represents the flight and delay\_type
    - Constraint: delay\_minutes >0
  - Allows for many-to-many relationships between flights and delay types

*saved\_preset*

- Represents a user-saved filter configuration
- Filters\_json stores the full filter parameters as a JSON string, which can be read and generated into a query when loading the preset
- created\_at and updated\_at track when the preset was created and last modified

## Relationships and Cardinalities

dim\_date **(1)** -> **(many)** fact\_flight

Each flight occurs at exactly one calendar date, while a single date can have many flights.

dim\_carrier **(1)** -> **(many)** fact\_flight

Each flight is marketed by one carrier, while a single carrier can have many flights

dim\_airport **(1)** -> **(many)** fact\_flight [origin]

Each flight has exactly one origin airport, while an airport can be an origin airport for many other flights.

dim\_airport **(1)** -> **(many)** fact\_flight [destination]

Each flight has exactly one destination airport, while an airport can be a destination airport for many other flights.

dim\_wac **(1)** -> **(many)** dim\_airport

Each airport belongs to one WAC region, while a WAC region can have many airports.

dim\_wac **(1)** -> **(many)** dim\_carrier

Each carrier is headquartered/registered in one WAC region, while a WAC region can have multiple carriers registered within it.

fact\_flight **(many)** <-> **(many)** dim\_delay\_type

A flight may have either zero or multiple causes for delay, and each delay type can apply to many flights

saved\_preset **(standalone)**

saved\_preset has no foreign key relationships to other entities. It exists as an independent store for user filter configurations for the dashboard. This does mean presets are globally accessible to all users of the application since this is designed as a single-instance analysis tool. All presets are shared and any user can create, view, edit, or delete them.

## Normalization

The original flight dataset we use contains repeated information about various airports, carriers, geographic regions, and dates for each flight row. Therefore, in order to eliminate any redundancy we went ahead and, in our schema, transformed the data into a fact table (*fact\_flight*) which only contains keys and measures while having other multiple dimension tables (*dim\_airport*, *dim\_carrier*, *dim\_wac*, *dim\_date*) which contain the descriptive attributes that depend on their primary keys. The delay causes are normalized further into *dim\_delay\_type* with the bridge table *fact\_flight\_delay* to aid in the many-to-many relationship between flights and delay types. In each table, all the non-key attributes depend only on the primary key and not on the other non-key attributes allowing for our schema to fall under the 3NF normalization.

### Verifying Schema is 3NF:

#### 1) FDs

##### **dim\_date**

*date\_id* -> *full\_date*  
*date\_id* -> *year*  
*date\_id* -> *quarter*  
*date\_id* -> *month*  
*date\_id* -> *day\_of\_month*  
*date\_id* -> *day\_of\_week*

##### **dim\_wac**

*wac* -> *wac\_name*  
*wac* -> *world\_area\_name*  
*wac* -> *country\_short\_name*  
*wac* -> *country\_code\_iso*  
*wac* -> *state\_code*  
*wac* -> *state\_name*  
*wac* -> *state\_fips*  
*wac* -> *is\_latest*

##### **dim\_airport**

*airport\_id* -> *airport*  
*airport\_id* -> *display\_airport\_name*  
*airport\_id* -> *display\_airport\_city\_name\_full*

*airport\_id* -> airport\_state\_code  
*airport\_id* -> airport\_state\_name  
*airport\_id* -> airport\_state\_fips  
*airport\_id* -> airport\_country\_name  
*airport\_id* -> airport\_country\_code\_iso  
*airport\_id* -> airport\_wac  
*airport\_id* -> latitude  
*airport\_id* -> longitude  
*airport\_id* -> airport\_is\_latest

#### **dim\_carrier**

*airline\_id* -> carrier  
*airline\_id* -> carrier\_name  
*airline\_id* -> unique\_carrier  
*airline\_id* -> unique\_carrier\_name  
*airline\_id* -> wac  
*airline\_id* -> carrier\_group  
*airline\_id* -> region

#### **dim\_delay\_type**

*delay\_type\_id* -> delay\_type\_name

#### **fact\_flight**

*flight\_id* -> date\_id  
*flight\_id* -> mkt\_carrier\_airline\_id  
*flight\_id* -> mkt\_carrier\_fl\_num  
*flight\_id* -> origin\_airport\_id  
*flight\_id* -> dest\_airport\_id  
*flight\_id* -> origin\_wac  
*flight\_id* -> dest\_wac  
*flight\_id* -> dep\_delay  
*flight\_id* -> arr\_delay  
*flight\_id* -> arr\_del15  
*flight\_id* -> cancelled  
*flight\_id* -> diverted  
*flight\_id* -> air\_time

*flight\_id* -> distance

**fact\_flight\_delay**

*(flight\_id, delay\_type\_id)* → delay\_minutes

## 2) Candidate Keys

dim\_date: date\_id

dim\_wac: wac

dim\_airport: airport\_id

dim\_carrier: airline\_id

dim\_delay\_type: delay\_type\_id

fact\_flight: flight\_id

fact\_flight\_delay: (flight\_id, delay\_type\_id)

## 3) Minimal Basis

The functional dependencies are already in minimal form because each dependency has an attribute on the right hand side and each left hand side is minimal as seen by either being a primary key or a composite key.

## 4) 3NF Relations

From the minimal basis there are these relations:

- **dim\_date** (date\_id, full\_date, year, quarter, month, day\_of\_month, day\_of\_week)
- **dim\_wac** (wac, wac\_name, world\_area\_name, country\_short\_name, country\_code\_iso, state\_code, state\_name, state\_fips, is\_latest)
- **dim\_airport** (airport\_id, airport, display\_airport\_name, display\_airport\_city\_name\_full, airport\_state\_code, airport\_state\_name, airport\_state\_fips, airport\_country\_name, airport\_country\_code\_iso, airport\_wac, latitude, longitude, airport\_is\_latest)
- **dim\_carrier** (airline\_id, carrier, carrier\_name, unique\_carrier, unique\_carrier\_name, wac, carrier\_group, region)
- **dim\_delay\_type** (delay\_type\_id, delay\_type\_name)
- **fact\_flight** (flight\_id, date\_id, mkt\_carrier\_airline\_id, mkt\_carrier\_fl\_num, origin\_airport\_id, dest\_airport\_id, origin\_wac, dest\_wac, dep\_delay, arr\_delay, arr\_del15, cancelled, diverted, air\_time, distance)

- **fact\_flight\_delay** (flight\_id, delay\_type\_id, delay\_minutes)

## Relational Schema

```
/*
  Datatypes:
    INT: integer or flag (0,1)
    BIGINT: larger whole number for bigger values than INT
    VARCHAR(n): essentially strings with length n
    DECIMAL(p,s): essentially like a double
    DATE: calendar date (year-month-day)
    TIMESTAMP: metadata (created_at, updated_at)
    TEXT: JSON for any configs present or string text of any
           length
*/

dim_date(
  date_id: INT [PK],
  full_date: DATE, year: INT,
  quarter: INT, month: INT,
  day_of_month: INT,
  day_of_week: INT)

dim_wac(
  wac: INT [PK],
  wac_name: VARCHAR(100),
  world_area_name: VARCHAR(100),
  country_short_name: VARCHAR(100),
  country_code_iso: VARCHAR(10),
  state_code: VARCHAR(10),
  state_name: VARCHAR(100),
  state_fips: VARCHAR(10),
  is_latest: INT)

dim_airport(
  airport_id: INT [PK],
  airport: VARCHAR(10),
  display_airport_name: VARCHAR(200),
  display_airport_city_name_full: VARCHAR(200),
  airport_state_code: VARCHAR(10),
  airport_state_name: VARCHAR(100),
  airport_state_fips: VARCHAR(10),
```

```
airport_country_name: VARCHAR(100),
airport_country_code_iso: VARCHAR(10),
airport_wac: INT [FK to dim_wac.wac],
latitude: DECIMAL(10,6),
longitude: DECIMAL(10,6),
airport_is_latest: INT)
```

```
dim_carrier(
    airline_id: INT [PK],
    carrier: VARCHAR(10),
    carrier_name: VARCHAR(200),
    unique_carrier: VARCHAR(10),
    unique_carrier_name: VARCHAR(200),
    wac: INT [FK to dim_wac.wac],
    carrier_group: VARCHAR(50),
    region: VARCHAR(50))
```

```
dim_delay_type(
    delay_type_id: INT [PK],
    delay_type_name: VARCHAR(50))
```

```
fact_flight(
    flight_id: BIGINT [PK],
    date_id: INT [FK to dim_date.date_id],
    mkt_carrier_airline_id: INT [FK to dim_carrier.airline_id],
    mkt_carrier_fl_num: INT,
    origin_airport_id: INT [FK to dim_airport.airport_id],
    dest_airport_id: INT [FK to dim_airport.airport_id],
    origin_wac: INT [FK to dim_wac.wac],
    dest_wac: INT [FK to dim_wac.wac],
    dep_delay: DECIMAL(8,2),
    arr_delay: DECIMAL(8,2),
    arr_del15: INT,
    cancelled: INT,
    diverted: INT,
    air_time: DECIMAL(8,2),
    distance: DECIMAL(10,2))
```

```
fact_flight_delay(
    flight_id: BIGINT [PK, FK to fact_flight.flight_id],
    delay_type_id: INT [PK, FK to dim_delay_type.delay_type_id],
    delay_minutes: DECIMAL(10,2))
```

```
saved_preset(
    preset_id: BIGSERIAL [PK],
    preset_name: VARCHAR(200),
    filters_json: TEXT,
```

**created\_at: TIMESTAMP,**  
**updated\_at: TIMESTAMP)**

